

Wren Vandervelde CS263 Project

Swift and Mobile-focused runtimes (working title)

Vision Statement

My goal for this project is to investigate the Swift language and runtime, specifically in respect to the language's use in mobile development. My first goal is to re-learn the details of Swift. I've worked with this language before, but I want to re-learn it with an eye towards what we've been learning in class, such as polymorphism and garbage collection. My next goal is to learn about the levels of bytecode that swift compiles into, such as Swift Intermediate Language (SIL) and the LLVM IL. I want to see what, if any, optimizations are made to aid in mobile execution. Finally, I want to investigate swift's cross-platform nature by running a number of benchmarks on different hardware, most likely my macbook, my iphone, and a larger machine such as CSIL.

Remark

These notes were made with the typst typesetting language and the ergo package.

Source: Swift Compiler | Swift.org

<https://www.swift.org/documentation/swift-compiler/>

Source: The LLVM Compiler Infrastructure

<https://llvm.org/>

Compilation pipeline:

- Parsing - Parse code into an AST with no type checking
- Semantic analysis - Type inference and type checking; augments the AST with type information
- Clang importer - import clang modules (*note: what are these?*) and attach C / Obj-C interfaces.
- SIL generation - convert the AST into SIL (Swift Intermediate Language). *note: look at the docs later*
- SIL guaranteed transformations - something something dataflow, something something correctness? look at this later
- SIL optimizations - optimize the SIL based on swift-specific patterns.
- LLVM IR generation - generate LLVM IR
- LLVM optimization - use LLVM Core's optimizer on the IR
- Machine Code Generation - use LLVM's code generation

Based on this pipeline, now I'm a bit confused because this looks an awful lot like a compiled language, which presumably wouldn't have a runtime? There are still IRs so the project still works, but I want to get this straight. It looks like swift is compiled, but there are still references to "the Swift Runtime" across the website.

Source: About Swift | Swift.org

<https://www.swift.org/about/>

"Memory is managed automatically" "Advanced control flow with do, guard, defer, and repeat keywords"

Quote: About Swift > Platform Support

Our goal is to provide source compatibility for Swift across all platforms, even though the actual implementation mechanisms may differ from one platform to the next. The primary example is that the Apple platforms include the Objective-C runtime, which is required to access Apple platform frameworks such as UIKit and AppKit. On other platforms, such as Linux, no Objective-C runtime is present, because it isn't necessary.

Source: The Swift Runtime - Your Silent Partner

<https://blog.jacobstechtaavern.com/p/the-swift-runtime-your-silent-partner>

The above isn't a very official source, but it does contain some information on what the swift runtime actually does, which is good. "The Swift Runtime, a.k.a libswiftCore." This pretty immediately dives into SIL, so I might need to come back to it.

Source: A Swift Tour

<https://docs.swift.org/swift-book/documentation/the-swift-programming-language/guidedtour/>

I figure before I learn SIL I should brush up on swift syntax and type system.

- Pretty standard typed language. Functions are first class
- type `String?` allows for `String` values or `nil`
- closure syntax: `{ number in 3 * number }` or
`{ (number: Int) -> Int in return 3 * number }`
or even shorter `numbers.sorted { $0 > $1 }`
- inside methods, instance var access can be unqualified or use `self`.
- `init` function is named `init`. (also `deinit` exists). multiple `inits` may share function signatures, but must have different arg names.
 - Initialization has quite a few rules to ensure safety
- `class Subclass : Superclass{...}`; use `super.whatever`, manually call `super.init`. must use `override` qualifier for polymorphism.
- custom getter and setters for “fake” instance var. also `willSet` and `didSet` for code that needs to run around assignment
- enums can have methods. Use `Type.case` or just `.case` when type is inferred
- structs are passed by value, classes are passed by reference
- `async`, `await`, `Task` blocks
- protocols are kinda like rust traits, extensions implement protocols for existing types
- errors implement `Error` protocol. functions that may throw an error are marked with `throws`.
- `try?` turns errors into optionals. `defer` plays nice with `throw`.
- generics use similar syntax to Rust

Source: SIL Documentation

<https://github.com/swiftlang/swift/blob/main/docs/SIL/SIL.md>

SIL has three representations:

- In memory: SIL is represented by data structures which are implemented in the compiler sources. Optimization passes use the in-memory representation of SIL.
- Textual: The compiler and related utilities can print and parse textual SIL files. Textual SIL files have the file extension `.sil`.
- Binary: SIL can be stored and read from binary files. Binary SIL files are called “swift-module” files and have the extension `.swiftmodule`. Note that the binary format is not stable. Swift-module files are not compatible between compiler versions.

SIL Documentation Continued:

- “A .sil file is a Swift source file with added SIL definitions.”
- OSSA (Ownership Static Single-Assignment): each variable is assigned once, and ownership is checked to statically find memory leaks or use after free.
- Raw SIL vs Canonical SIL. Raw SIL may have dataflow errors and isn’t optimized yet.
- SIL types start with \$, SIL functions start with @, SIL values start with %
- SIL converts functions to basic blocks
 - all basic blocks end with a terminator
 - basic blocks take arguments
 - function arguments
 - forwarded arguments from the previous terminator
 - “phi arguments” these apparently have something to do with LLVM’s phi nodes
 - in OSSA form, all arguments have ownership annotations
- Ownership types (also starts with @, don’t confuse them with functions)
 - @owned: freestanding value is consumed exactly once during the function (by storing or destroying it typically)
 - @guaranteed: value that depends on another value’s existence, such as a borrow.
 - @unowned: “A value that is only guaranteed to be instantaneously valid.” Must be moved into @owned or @guaranteed being consumed.
 - Trivial values (such as int) don’t have ownership

Next Steps

This section is a loose collection of “things I want to look at later.”

definitely look at:

- [SIL docs](#)
- memory management
- “The Swift Runtime - Your Silent Partner”
- “Advanced control flow with do, guard, defer, and repeat keywords” - About Swift
- optional types
- LLVM IR
- SIL guaranteed transformations

less important:

- SIL optimizations
- LLVM optimizations